

Math/Comp 225
Homework 10
Fall 2025

Instructions: See syllabus for detailed instructions and rules about what and whom you can/cannot consult to complete the assignment. If you work together on this assignment, you **MUST** indicate on your submission with whom you worked (and on which subset of problems), and the answers, code, experimental results must all be in your own words.

1. There is a well-known predator-prey model that is expressed as

$$\begin{aligned}y_1' &= y_1(\alpha_1 - \beta_1 y_2) \\ y_2' &= y_2(-\alpha_2 + \beta_2 y_1)\end{aligned}$$

The $y_1(t)$ is “prey”, $y_2(t)$ is “predator”. Here, α_1, α_2 are the natural birth and death rates (in isolation) of the respective populations, and β_1, β_2 give the effect of interactions between the two populations.

Let's set $\alpha_1 = 1, \beta_1 = .1, \alpha_2 = .5; \beta_2 = 0.02$, and take $y_1(0) = 100, y_2(0) = 10$.

This is a nonlinear problem of the form $\mathbf{y}'(t) = \mathbf{f}(t, \mathbf{y}), \mathbf{y}(0) = \mathbf{y}_0$.

Find $\mathbf{J}_f(t)$, the Jacobian of $\mathbf{f}$ with respect to $\mathbf{y}$ and evaluate it at $t = 0$. Do we have the real part of the eigenvalues of $\mathbf{J}_f(0)$ less than 0? (if yes, at least near $t = 0$, this is a well-conditioned ODE).

Implement backward Euler and apply it to this problem for $t = 0$ to $t = 25$. Plot the two populations as functions of time. What behavior do you observe? Try varying the initial populations and see if you can find scenarios where one or the other becomes extinct, or where the two populations reach steady state.

2. Euler methods evaluate f once per step and have order 1. What if we sample f twice per step? Can we get a 2nd order method? The answer is yes, if we do it right. Let

$$\begin{aligned}k_1 &= hf(t_n, y_n) \\ k_2 &= hf(t_n + \alpha h, y_n + \beta k_1) \\ y_{n+1} &= y_n + ak_1 + bk_2.\end{aligned}$$

There are 4 parameters α, β, a, b and we want to choose them so the local truncation error is $O(h^3)$.

Use the Taylor series:

$$y(t_{n+1}) = y(t_n) + y'(t_n)h + y''(t_n)h^2/2 + O(h^3)$$

and the facts that $y'(t) = f(t, y)$ and $y''(t) = f_t + f_y f$ (the subscripts mean partial with respect to t and y , respectively), to come up with system of (3) equations that must be satisfied so that the LTE will be $O(h^3)$. There are 4 unknowns, so there's no unique solution, but if you take that one that has $a = b$ and $\alpha = \beta$, you will derive the Runge-Kutta order 2 method. (Hint, you'll need to use TS expansion of k_2 , as well.)

3. Here, we investigate one way to get a multistep formula. Assume

$$y_{n+1} = \alpha y_n + h(\beta_1 f(t_n, y_n) + \beta_2 f(t_{n-1}, y_{n-1})),$$

and α, β_1, β_2 are to be determined from 3 equations. To get the 3 equations: First, assume that $y(t) = 1$, which means $y'(t) = 0$ (i.e. $f(t_{n-1}, y_{n-1}) = 0$). We get a first equation by making the rule exact for $y(t) = 1$ (e.g. $1 = \alpha \cdot 1 + h(\beta_1 \cdot 0 + \beta_2 \cdot 0)$). Repeat the process of forcing the formula to be exact for $y(t) = t$ (i.e. $y'(t) = 1 = f(t_{n-1}, y_{n-1})$), and again for $y(t) = t^2$.

- Write the 3 equations that force the rule to be exact.
 - They must hold for any value of t_n , so in particular let $t_{n-1} = 0$, $h = 1$ (note, this choice defines t_n and t_{n+1}) and solve for α, β_1, β_2 .
 - Verify that you get the explicit 2-step method $y_{n+1} = y_n + \frac{h}{2}(3f(t_n, y_n) - f(t_{n-1}, y_{n-1}))$.
 - Check if the method is consistent (i.e. the coefficients satisfy the conditions of the theorem given in class).
4. Repeat the procedure in the previous problem, except use it to derive the coefficients in the implicit 2-step method

$$y_{n+1} = \alpha y_n + h(\beta_0 f(t_{n+1}, y_{n+1}) + \beta_1 f(t_n, y_n)).$$

Here, you will want to assume $t_n = 0$, and $h = 1$ (so $t_{n+1} = 1$). Verify that you get the Trapezoid method.

5. We will use the previous 2 formulas as a so-called 'predictor-corrector' (PECE) scheme on the following IVP:

$$y'(t) = -2ty(t), y(0) = y_0$$

The exact solution is $y(t) = \frac{1}{1+t^2}$.

- Predict y_2 using the explicit method $y_2 = y_1 + \frac{h}{2}(3f(t_1, y_1) - f(t_0, y_0))$ (you will need to choose a method to first find y_1 here, be sure to say what you used).
 - Correct that guess: substitute the value you just got in for y_2 in the right-hand side of the implicit Trapezoid rule, then update to get your new estimate of y_2 .
 - Compare the two estimates of y_2 to $y(t_2)$.
 - Repeat the procedure to obtain a prediction of y_3 , a correction to y_3 and compare to $y(t_3)$.
6. Consider the linear problem $\mathbf{y}' = \mathbf{A}\mathbf{y}$, $\mathbf{y}(0) = \mathbf{y}_0$ for $n \times n$ matrix $\mathbf{A}$. Assume that $\mathbf{A}$ is diagonalizable and that $\text{Re}(\lambda_i) < 0$ for $i = 1, \dots, n$.

Assume you are using the Trapezoid method to determine $\mathbf{y}_n$. Find the magnification factor (this will be a matrix, $\mathbf{M}$), ie. $\mathbf{y}_{n+1} = \mathbf{M}\mathbf{y}_n$. We will want $\rho(\mathbf{M}) < 1$. Will this be true for any (positive) h , under the assumptions on $\text{Re}(\lambda_i) < 0$? Explain your reasoning.

① We have a predator-prey model

(1) prey: $y_1' = y_1(d_1 - \beta_1 y_2)$ d_i are birth/death rates
predator: $y_2' = y_2(-d_2 + \beta_2 y_1)$ β_i is effect of interactions

Let's set $d_1 = 1$, $\beta_1 = 0.1$, $d_2 = 0.5$, $\beta_2 = 0.02$ and take $y_1(0) = 100$ and $y_2(0) = 10$.

(a) Find $J_f(t)$ wrt y and evaluate at $t = 0$. Are the real parts of the eigenvalues of $J_f(0)$ less than zero?

Soln: The Jacobian of the system (1) takes the form

$$[J_f(t)]_{ij} = \frac{\partial y_i'}{\partial y_j} \Rightarrow J_f(t) = \begin{bmatrix} d_1 - \beta_1 y_2(t) & -\beta_1 y_1(t) \\ y_2(t) \beta_2 & -d_2 + \beta_2 y_1(t) \end{bmatrix} \quad (2)$$

The parameters provided correspond to an ODE system

$$\vec{f}(t, y) = \vec{y}'(t) = \begin{bmatrix} y_1(t)(1 - 0.1 y_2(t)) \\ y_2(t)(-0.5 + 0.02 y_1(t)) \end{bmatrix} ; \vec{y}_0 = \begin{bmatrix} 100 \\ 10 \end{bmatrix}$$

Using our initial condition $\vec{y}_0$ together w/ (2), we see that

$$J_f(0) = \begin{bmatrix} 0 & -10 \\ 0.2 & 1.5 \end{bmatrix}.$$

Now we compute the eigenvalues via the characteristic polynomial

$$\det(J_f(0) - \lambda I) = (-\lambda)(1.5 - \lambda) + 2 = \lambda^2 - \frac{3}{2}\lambda + 2$$

$$\Rightarrow \lambda = \frac{\frac{3}{2} \pm \sqrt{\frac{9}{4} - 8}}{2} \Rightarrow \operatorname{Re}(\lambda) = \frac{3}{4}$$

So this ODE system is not well-conditioned at $t = 0$.

(b) Implement backward Euler and apply it to this problem for $t \in [0, 25]$. Plot the populations as functions of time.

Soln: See attached R code

② We'll derive a 2nd order, 2-step method. Define

$$k_1 = hf(t_n, y_n) \quad (1)$$

$$k_2 = hf(t_n + \alpha h, y_n + \beta k_1) \quad (2)$$

$$y_{n+1} = y_n + ak_1 + bk_2 \quad (3)$$

We want to choose α, β, a, b to achieve local truncation error $O(h^3)$.

Use the Taylor Series $y(t_{n+1}) = y(t_n) + y'(t_n)h + y''(t_n)\frac{h^2}{2} + O(h^3)$ and the facts that $y'(t) = f(t, y)$ and $y''(t) = f_x + f_y f$ to come up with a system of 3 equations that must be satisfied so that the LTE will be $O(h^3)$.

Soln: The Taylor Series

$$y(t_{n+1}) = y(t_n) + y'(t_n)h + y''(t_n)\frac{h^2}{2} + O(h^3)$$

is precisely the statement we want to hold: LTE for our approximation to $y(t)$ is $O(h^3)$. Using (3), we can relate this to the system we defined by writing

$$y_n + ak_1 + bk_2 = y(t_n) + y'(t_n)h + y''(t_n)\frac{h^2}{2} + O(h^3)$$

As $y_n = y(t_n)$ by definition, then we want to satisfy

$$ak_1 + bk_2 = y'(t_n)h + y''(t_n)\frac{h^2}{2} + O(h^3). \quad (*)$$

Now, recall from lecture that we can write

$$y'(t) = f(t, y(t)) \implies y''(t) = f_x(t, y(t)) + f_y(t, y(t)) \cdot f(t, y(t))$$

(chain rule)

$$= f_x + f_y \cdot f \quad (4)$$

With this, also consider the linear Taylor expansion of k_2 :

$$\begin{aligned} f(t_n + \alpha h, y_n + \beta k_1) &= f(t_n, y_n) + \alpha h f_x(t_n, y_n) + \beta k_1 f_y(t_n, y_n) \\ &= f + \alpha h f_x + \beta h f \cdot f_y + O(h^2) \\ &= f + h(\alpha f_x + \beta f \cdot f_y) + O(h^2) \quad (**)$$

From **(*)** we see that

$$h_2 = hf(t_n + \alpha h, y_n + \beta h_1) = fh + h^2(\alpha f_x + \beta f \cdot f_y) + O(h^3)$$

$$[f(t, y) = y'(t)] = y'(t_n)h + h^2(\alpha f_x + \beta f \cdot f_y) + O(h^3)$$

Now we refer back to the LHS of **(*)** and see that

$$ah_1 + bh_2 = ahf(t_n, y_n) + b[y'(t_n)h + h^2(\alpha f_x + \beta f \cdot f_y) + O(h^3)]$$

$$[f(t, y) = y'(t)] = (a+b)y'(t_n)h + bh^2(\alpha f_x + \beta f \cdot f_y) + O(h^3)$$

Hence if $a+b=1$ and $\alpha h = \beta h = \frac{1}{2}$, we use $y''(t) = f_x + f_y f$ to obtain

$$ah_1 + bh_2 = y'(t_n)h + \frac{h^2}{2}y''(t_n) + O(h^3)$$

precisely as desired. So the method is given by **(1) - (3)** together with the highlighted constraints. Finally, by choosing $\alpha = \beta = 1$ and $a = b = \frac{1}{2}$, we obtain

$$y_{n+1} = y_n + \frac{h}{2} [f(t_n, y_n) + f(t_n, y_n + hf(t_n, y_n))] + O(h^3),$$

which is the RK-2 method we derived in class.

③ Assume

$$y_{n+1} = \alpha y_n + h[\beta_1 f(t_n, y_n) + \beta_2 f(t_{n-1}, y_{n-1})]$$

and α, β_1, β_2 are to be determined from three equations. To get the three equations:

- assume $y(t) = 1 \Rightarrow y'(t) = 0 \Rightarrow f(t_{n-1}, y_{n-1}) = 0$
- Make the rule exact for $y(t) = 1$, i.e.
$$1 = \alpha \cdot 1 + h(\beta_1 \cdot 0 + \beta_2 \cdot 0) \quad (1)$$
- repeat for $y(t) = t$ and $y(t) = t^2$.

Soln: We want to force exactness for both

$$y(t) = t \Rightarrow y'(t) = f(t, y) = 1$$

and

$$y(t) = t^2 \Rightarrow y'(t) = f(t, y) = 2t.$$

Writing $y(t_n) = y_n$, then the former yields

$$t_{n+1} = \alpha t_n + h[\beta_1 \cdot 1 + \beta_2 \cdot 1] \quad (2)$$

and the latter yields

$$t_{n+1}^2 = \alpha t_n^2 + h[\beta_1 \cdot 2t_n + \beta_2 \cdot 2t_{n-1}] \quad (3)$$

So (1) - (3) are the equations we want to satisfy $\square$

Solve for α, β_1 and β_2 to get an explicit, two-step method. Check if the method is consistent.

Soln: Since these must hold for any t_n , then choose $t_{n-1} = 0$ and $h=1$. We also know that $\alpha=1$ from (1).

$$\therefore (3) \Rightarrow t_{n+1}^2 = \alpha t_n^2 + h[\beta_1 \cdot 2t_n + \beta_2 \cdot 2t_{n-1}]$$

$$2^2 = 1 \cdot 1^2 + 1 \cdot 2[\beta_1 \cdot 1 + \beta_2 \cdot 2 \cdot 0]$$

$$4 = 1 + 2\beta_1 \Rightarrow \beta_1 = \frac{3}{2}$$

$$(2) \Rightarrow t_{n+1} = \alpha t_n + h[\beta_1 \cdot 1 + \beta_2 \cdot 1]$$

$$h = h\left[\frac{3}{2} + \beta_2\right] \quad [\text{since } \alpha=1 \text{ and } t_{n+1} - t_n = h]$$

$$1 = \frac{3}{2} + \beta_2 \Rightarrow \beta_2 = -\frac{1}{2}$$

Therefore, we do in fact obtain $\alpha = 1$, $\beta_1 = \frac{3}{2}$ and $\beta_2 = -\frac{1}{2}$ to give the desired two-step method.

For consistency, recall that a general multistep method takes the form

$$y_{n+1} = \sum_{j=0}^p a_j y_{n-j} + h \sum_{j=-1}^p b_j f(t_{n-j}, y_{n-j})$$

and that the consistency criterion holds iff

$$\sum_{j=0}^p a_j = 1 \quad \text{and} \quad -\sum_{j=0}^p j a_j + \sum_{j=-1}^p b_j = 1$$

For our explicit, two-step method, we have $p=2$ with

$$a_0 = 1, b_{-1} = 0, b_0 = \frac{3}{2}, b_1 = -\frac{1}{2}.$$

The condition $\sum_{j=0}^p a_j = 1$ holds by inspection, and

$$-\sum_{j=0}^p j a_j + \sum_{j=-1}^p b_j = 0 + 0 + \frac{3}{2} - \frac{1}{2} = 1,$$

so the consistency criterion holds.

④ Repeat the procedure in Q3, except do it for the implicit, two-step method

$$y_{n+1} = \alpha y_n + h[\beta_0 f(t_{n+1}, y_{n+1}) + \beta_1 f(t_n, y_n)].$$

Verify that you get the trapezoid method.

Soln: Once again, we'll force exactness for

$$y(x) = 1 \Rightarrow y'(x) = f(x, y) = 0 \Rightarrow \alpha = 1 \quad (1)$$

$$\begin{aligned} y(x) = x &\Rightarrow y'(x) = f(x, y) = 1 \\ &\Rightarrow x_{n+1} = x_n + h(\beta_0 + \beta_1) \end{aligned} \quad (2)$$

$$\begin{aligned} y(x) = x^2 &\Rightarrow y'(x) = f(x, y) = 2x \\ &\Rightarrow x_{n+1}^2 = \alpha x_n^2 + h(\beta_0 \cdot 2x_{n+1} + \beta_1 \cdot 2x_n) \end{aligned} \quad (3)$$

Now we'll assume that $x_n = 0$ and $h = 1 \Rightarrow x_{n+1} = 1$. Using $\alpha = 1$ via (1), we have:

$$\begin{aligned} (3) &\Rightarrow x_{n+1}^2 = \alpha x_n^2 + h(\beta_0 \cdot 2x_{n+1} + \beta_1 \cdot 2x_n) \\ 1^2 &= 1 \cdot 0^2 + 1 \cdot (\beta_0 \cdot 2 \cdot 1 + \beta_1 \cdot 2 \cdot 0) \end{aligned}$$

$$1 = 2\beta_0 \Rightarrow \beta_0 = \frac{1}{2}$$

$$(2) \Rightarrow x_{n+1} = x_n + h(\beta_0 + \beta_1)$$

$$h = h(\beta_0 + \beta_1)$$

$$1 = (\beta_0 + \beta_1) \Rightarrow \beta_0 = \beta_1 = \frac{1}{2}$$

Therefore, we obtain

$$y_{n+1} = y_n + \frac{h}{2} [f(t_{n+1}, y_{n+1}) + f(t_n, y_n)],$$

which is in fact the trapezoid method $\square$

⑤ We'll use our results in Q3 and Q4 for a Predictor-Corrector scheme for the IVP

$$y' = -2t \cdot y^2; y(0) = y_0$$

The exact solution is $y(t) = 1/(1+t^2)$

(a) Predict y_2 using the explicit step $y_2 = y_1 + \frac{h}{2}[3f(t_1, y_1) - f(t_0, y_0)]$.

Soln: First, we'll determine y_1 via a single forward Euler step and $y_0 = 1$.

$$y_1 = y_0 + h f(t_0, y_0) = 1 + h \cdot 0 = 1$$

Now we have

$$\begin{aligned} y_2 &= y_1 + \frac{h}{2}[3f(t_1, y_1) - f(t_0, y_0)] \\ &= 1 + \frac{h}{2}[3 \cdot f(h, 1) - f(0, 1)] \\ &= 1 + \frac{h}{2}[3 \cdot (-2h \cdot 1^2)] \\ &= 1 + \frac{h}{2} \cdot -6h = 1 - 3h^2 \end{aligned}$$

(b) Correct your guess: Use your estimate to y_2 (above) on the RHS of the trapezoid rule to obtain a new estimate to y_2 .

Soln: We'll denote the corrected guess as $\hat{y}$. We have

$$\begin{aligned} \hat{y}_2 &= y_1 + \frac{h}{2}[f(t_2, y_2) + f(t_1, y_1)] \\ &= 1 + \frac{h}{2}[f(2h, 1-3h^2) + f(h, 1)] \\ &= 1 + \frac{h}{2}[(-2 \cdot 2h \cdot (1-3h^2)^2) + 3 \cdot (-2h \cdot 1^2)] \\ &= 1 + \frac{h}{2}[(-4h + 24h^3 - 36h^5) + (-6h)] \\ &= 1 - 5h^2 + 12h^4 - 18h^6 \quad \square \end{aligned}$$

(c) Compare y_2 and $\hat{y}_2$ to $y(t_2)$.

Soln: This particular soln $y(t)$ has a well-known Taylor Series expansion. Expanding $y(t)$ on $[0, t]$ we have

$$y(t) = 1 - t^2 + t^4 - \dots + \dots$$

Taking $t = t_2 = 2h$, we obtain $y(t_2) = 1 - 4h^2 + 16h^4 - 64h^6 + O(h^8)$.

Thus, we see that the error in our initial approximation is

$$y(t_2) - y_2 = 1 - 4h^2 + O(h^4) - (1 - 3h^2) = -h^2 + O(h^4)$$

and the error in our corrected approximation is

$$\begin{aligned} y(t_2) - \hat{y}_2 &= 1 - 4h^2 + 10h^4 - 64h^6 + O(h^8) - (1 - 5h^2 + 12h^4 - 18h^6) \\ &= -h^2 + 4h^4 - 40h^6 + O(h^8). \end{aligned}$$

So, the corrected estimate is order-8 accurate, and the original is order-4.

(d) Repeat this procedure for t_3 .

Soln: For the explicit step, we'll use

$$y_1 = 1 \quad \text{and} \quad y_2 = \hat{y}_2 = 1 - 5h^2 + 12h^4 - 18h^6$$

Hence, we have

$$y_3 = \hat{y}_2 + \frac{h}{2} [3 \cdot f(t_2, \hat{y}_2) - f(t_1, y_1)]$$

such that $f(t_1, y_1) = f(h, 1) = (-2h \cdot 1^2) = -2h$ and

$$f(t_2, \hat{y}_2) = f(2h, 1 - 5h^2 + 12h^4 - 18h^6)$$

$$= -2 \cdot 2h \cdot (1 - 5h^2 + 12h^4 - 18h^6)^2$$

$$= -4h + 40h^3 - 144h^5 + 624h^7 - 1296h^9 + 1728h^{11} - 1296h^{13}.$$

[Used calculator on Wolfram Alpha here]

$$\therefore y_3 = 1 - 5h^2 + 12h^4 - 18h^6 + \dots$$

$$\dots + \frac{h}{2} [3 \cdot (-4h + 40h^3 - 144h^5 + 624h^7 - 1296h^9 + 1728h^{11} - 1296h^{13}) + 2h]$$

$$= 1 - 10h^2 + 72h^4 - 312h^6 + 936h^8 - 1944h^{10} + 2592h^{12} - 1944h^{14}$$

Now we compute the corrected estimate via trapezoid rule:

$$\hat{y}_3 = y_2 + \frac{h}{2} [f(t_3, y_3) + f(t_2, \hat{y}_2)] \quad \text{s.t.}$$

$$f(t_3, y_3) = f(3h, 1 - 10h^2 + 72h^4 - 312h^6 + 936h^8 - 1944h^{10} + 2592h^{12} - 1944h^{14})$$

We see that y_3 is an order-10 approximation and, using the above Taylor Series, $\hat{y}_3$ is an order-12 approximation.

Since Taylor's Thm holds on (a, b) for an expansion $f: [a, b] \rightarrow \mathbb{R}$, then we expect the PECE scheme to be highly accurate for $h \ll 1$.

⑥ Consider $\vec{y}' = A\vec{y}$; $\vec{y}(0) = \vec{y}_0$ for A an $n \times n$ matrix. Assume that A is diagonalizable and that $\operatorname{Re}(\lambda_i) < 0$ for $i = 1, \dots, n$.

Assume you are using the trapezoid method to determine $\vec{y}_n$. Find the magnification factor M s.t. $\vec{y}_{n+1} = M\vec{y}_n$.

Soln: Using the trapezoid method, then we have

$$\begin{aligned}\vec{y}_{n+1} &= \vec{y}_n + \frac{h}{2} (A\vec{y}_{n+1} + A\vec{y}_n) \\ \vec{y}_{n+1} - \frac{h}{2} A\vec{y}_{n+1} &= \vec{y}_n + \frac{h}{2} A\vec{y}_n \\ (I - \frac{h}{2} A)\vec{y}_{n+1} &= (I + \frac{h}{2} A)\vec{y}_n \\ \Rightarrow M &= (I - \frac{h}{2} A)^{-1} (I + \frac{h}{2} A) \quad \square\end{aligned}$$

We will want $\rho(M) < 1$. Will this be true for any $h > 0$ under the assumption that $\operatorname{Re}(\lambda_i) < 0$. Explain.

Soln: Since A is assumed to be diagonalizable, then $\exists$ an invertible matrix P such that $A = P\Lambda P^{-1}$ where λ_i are along the diagonal of Λ . However, for any choice of P we may diagonalize the identity as

$$I = PIP^{-1},$$

which implies that we can write

$$I - \frac{h}{2} A = PIP^{-1} - \frac{h}{2} P\Lambda P^{-1} = P(I - \frac{h}{2} \Lambda)P^{-1}$$

Since $I - \frac{h}{2} \Lambda$ is a diagonal matrix, then

$$(I - \frac{h}{2} A)^{-1} = (P(I - \frac{h}{2} \Lambda)P^{-1})^{-1} = (P^{-1})^{-1} (I - \frac{h}{2} \Lambda)^{-1} P^{-1} = P\Lambda^{-1}P^{-1}$$

such that $(\Lambda^{-1})_{ii} = 1/(1 - \frac{h}{2} \lambda_i)$. So, now we have

$$\begin{aligned}M &= (I - \frac{h}{2} A)^{-1} (I + \frac{h}{2} A) = (P\Lambda^{-1}P^{-1})(P(I + \frac{h}{2} \Lambda)P^{-1}) \\ &= P\Lambda^{-1}\Lambda^+P^{-1}\end{aligned}$$

such that $(\Lambda^+)_{ii} = 1 + \frac{h}{2} \lambda_i$. So, we have shown that M is diagonalizable, and that its eigenvalues are of the form

$$(\Lambda^{-1}\Lambda^+)_{ii} = \frac{1 + \frac{h}{2} \lambda_i}{1 - \frac{h}{2} \lambda_i}.$$

Now let $\frac{h}{2} \lambda_i = z_i \in \mathbb{C}$. For all i we want

$$\left| \frac{1+z_i}{1-z_i} \right| < 1 \iff |1+z_i| < |1-z_i| \iff |1+z_i|^2 < |1-z_i|^2$$

$$\therefore |1+z_i|^2 < |1-z_i|^2$$

$$|1+2z_i+z_i^2| < |1-2z_i+z_i^2|$$

Since we have a strict inequality, we can apply the Δ -ineq to either side of the inequality w/out altering the strict inequality:

$$1 + 2|z_i| + |z_i|^2 > 1 - 2|z_i| + |z_i|^2 \quad [\Delta\text{-ineq}]$$

$$2|z_i| > -2|z_i|$$

$$|z_i| + |\bar{z}_i| > -|z_i| - |\bar{z}_i|$$

$$[|z| = |\bar{z}| \text{ for } z \in \mathbb{C}]$$

$$|z_i + \bar{z}_i| > -|z_i + \bar{z}_i|$$

$$[\Delta\text{-ineq}]$$

$$\Leftrightarrow 2\operatorname{Re}(z_i) > -2\operatorname{Re}(z_i)$$

$$[z + \bar{z} = 2\operatorname{Re}(z)]$$

$$\Leftrightarrow \operatorname{Re}(z_i) > 0.$$

So we have $\rho(M) < 1 \Leftrightarrow \operatorname{Re}(z_i) > 0$. Since $\operatorname{Re}(z_i) = \operatorname{Re}(\frac{h}{2}\lambda_i) = \frac{h}{2}\operatorname{Re}(\lambda_i)$ and $\operatorname{Re}(\lambda_i) < 0$ by assumption, then $\rho(M) < 1$ holds for $h > 0$. $\square$

Homework 10, Question 1

Christian Lentz

2025-12-07

Question 1

The Backward Euler iteration for a system of ODEs takes the form

$$\vec{y}_{i+1} = \vec{y}_i + h\vec{f}(t_{i+1}, \vec{y}_{i+1}).$$

Since the method is implicit, we need to solve a nonlinear system at each time step using Newton's Method. Write $\vec{y}_{i+1} = \vec{x} = [x_1, x_2]^\top$. Thus, we are trying to solve

$$\vec{g}(\vec{x}) = \vec{x} - \vec{y}_i - h\vec{f}(t_{i+1}, \vec{x}) = \vec{0}$$

via the Newton iteration

$$J_g(\vec{x}_k)(\vec{x}_{k+1} - \vec{x}_k) = -g(\vec{x}_k).$$

For our specific problem, we have

$$\vec{f}(t_{i+1}, \vec{x}) = [x_1(1 - 0.1x_2), x_2(-0.5 + 0.02x_1)]^\top.$$

Since we are treating $\vec{y}_i$ as a constant here, then we see that we have

$$J_g(\vec{x}) = J(\vec{x} - h\vec{f}(t_{i+1}, \vec{x})) = I - hJ_f(t_{i+1}, \vec{x})$$

Pseudocode

With this in mind, here is pseudocode for my implementation. This describes a general purpose implementation, but the code that I will write will be adapted specifically for the example.

Given: a system of ODEs $\vec{y}' = \vec{f}(t, \vec{y})$, an interval $t \in [0, a]$ and an initial condition $\vec{y}_0$

Return: a data frame which holds solutions to each ODE of the system across a discretization of $t \in [0, a]$

1. Let T be a (uniform) discretization $\{0 = t_0, t_1, \dots, t_{n-1}, t_n = a\}$ of $t \in [0, a]$ such that $t_{i+1} - t_1 = h$.
2. Let $\vec{x} = \vec{y}(t_1)$
3. For each i corresponding to $t_i \in T \setminus t_0$ **DO**:
 - Use Forward Euler to compute $x_0 = x + h \cdot \vec{f}(t_i, \vec{x})$
 - Let $\vec{g}(\vec{x}) = \vec{x} - \vec{y}_{i-1} - h\vec{f}(t_i, \vec{x})$

- Compute the Jacobian $J_g(\vec{x})$
- Using $J_g(\vec{x})$, solve $\vec{g}(\vec{x}) = \vec{0}$ via Newton's Method (see above discussion) to obtain $\vec{x}_*$
- Record approximated solution $\vec{y}_{i+1} = \vec{y}(\vec{x}_*)$
- Update $\vec{x} = \vec{y}_{i+1}$

3. Return

For this particular problem, we have $a = 25$.

Code

The following code chunk implements helper functions.

```
# birth/death rate, interaction terms, size of system, discretization parameter
a1 <- 1
b1 <- 0.1
a2 <- 0.5
b2 <- 0.02
n <- 2
h <- 0.1

# Define the ODE system. We use this to apply a single step of forward Euler
# in order to get initial guesses for each of our Newton iterations.
ff <- function(x) {
  f1 <- x[1]*(a1 - b1*x[2])
  f2 <- x[2]*(-a2 + b2*x[1])
  return(c(f1, f2))
}

# Define the Jacobian for this specific system
J <- function(x) {
  Jf <- cbind( c(a1 - b1*x[2], x[2]*b2), c(-b1*x[1], -a2 + b2*x[1]) )
  I <- diag(1, n, n)
  return(I-h*Jf)
}

# Define: the p-norm of a vector
# Defaults to p=2 if not specified
vnorm = function(v,p=2) {
  # infinity norm
  if ( p == "I" ) {
    return(max(abs(v)))
  }
  else {
    return(sum(abs(v)^p)^(1/p))
  }
}

# Implement Newton's Method for this specific problem
# INPUT:
# - the system `FF` and its Jacobian `J`
# - an initial guess `x0`
# RETURN: 'estimates' which is a matrix whose columns contain our estimates at
```

```

# each iteration; iterate increases with column index
NM <- function(FF, J, x0, max.its=20, tol=0.5*10^-5) {
  estimates <- x0
  errors <- c(vnorm(FF(x0)))
  xk <- x0
  # iterate until:
  # (1) we are sufficiently close to soln OR
  # (2) we have more iterations to complete:
  i <- 1
  while ( vnorm(FF(xk)) > tol && i <= max.its ) {
    # evaluate function
    Fx <- FF(xk)
    # get Jacobian
    Jx <- J(xk)
    # solve system
    sk <- solve(Jx, -Fx)
    # update data with computed step
    new <- xk + sk
    estimates <- cbind(estimates, new)
    errors[i+1] <- vnorm(FF(new))
    # update for new iteration
    xk <- new
    i <- i+1
  }
  df.estimates <- data.frame(estimates)
  df.errors <- data.frame(errors)
  return( list(est = df.estimates, err = df.errors, final = xk) )
}

```

The code chunk below implements the Backward Euler iteration using the above helper functions.

```

# Implement the backward Euler iteration. User must provide
# - time-step, a, at which we terminate the iteration.
# - an initial condition (vector) y0
BE <- function(a, y0) {
  # discretize temporal domain
  tt <- seq(0, a, by=h)
  TT <- length(tt)
  # sequence of estimates to be returned
  estimates <- y0
  # loop variable
  y_curr <- y0 # solution from most recent Newton iteration
  x <- y_curr + h * ff(y_curr) # seeds the next iteration
  # main loop
  for (i in 2:TT) {
    # use ff to define the nonlinear system that we solve at this iterate
    GG <- function(y) {
      return(y - y_curr - h*ff(y))
    }
    # nonlinear solve via Newton
    y_next <- NM (
      GG, # the system
      J, # the Jacobian
      x, # a guess to seed the iteration; obtain via FE
    )
  }
}

```

```

    max.its = 2      # two iterations at most
  )
  # update estimates and loop variable
  estimates <- cbind(estimates, y_next$final)
  y_curr <- y_next$final
  # NOTE: a single FE step to determine x to seed the next Newton iteration
  x <- y_curr + h * ff(y_curr)
}
df <- data.frame(estimates)
return(df)
}

```

Plots

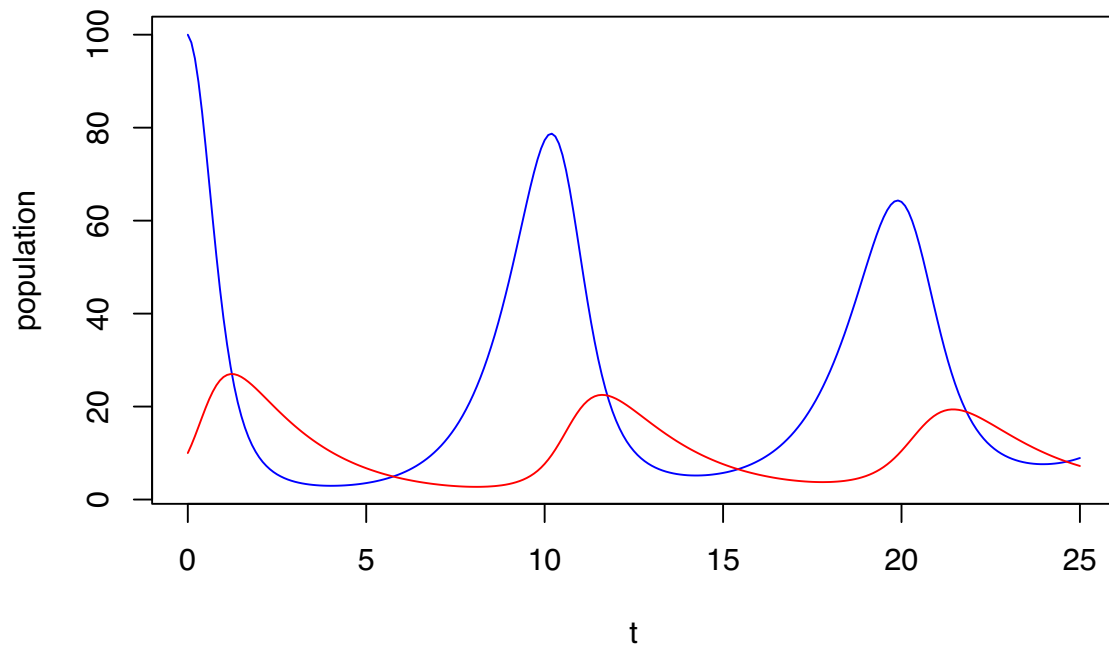
First, observe that the two populations have an inverse oscillatory relationship. By this, I mean that each population oscillates over time. As the prey curve approaches a local maximum, the predator curve is beginning to recover from hitting a local minimum. Similarly, as the prey curve approaches a local minimum, the predator curve begins to decrease after hitting a local maximum. In the context of this problem, this makes a lot of sense; when food (prey) is scarce, the predator population dies-off, and the opposite happens when food (prey) is abundant.

```

# [0, 25] with initial condition (100, 10)
df1 <- BE(25, c(100, 10))
tt <- seq(0, 25, by=h)
plot(tt, df1[1,], type='l', xlab="t", ylab="population", col='blue', main="Predator (red) v. Prey (blue)")
lines(tt, df1[2,], col='red')

```

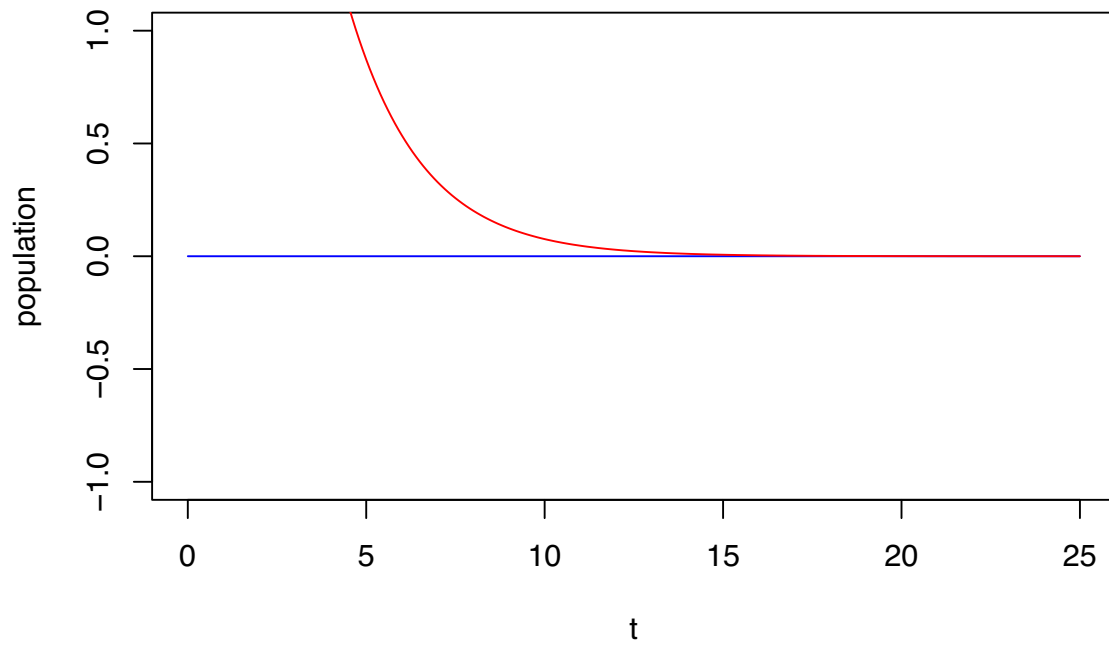
Predator (red) v. Prey (blue) Population over Time



After some experimentation, it seems that I can only get a population to go extinct in the case that the initial condition already assumes that the other population is extinct. Even in extreme cases (see the second plot below) where you expect a population to go extinct, the two still oscillate.

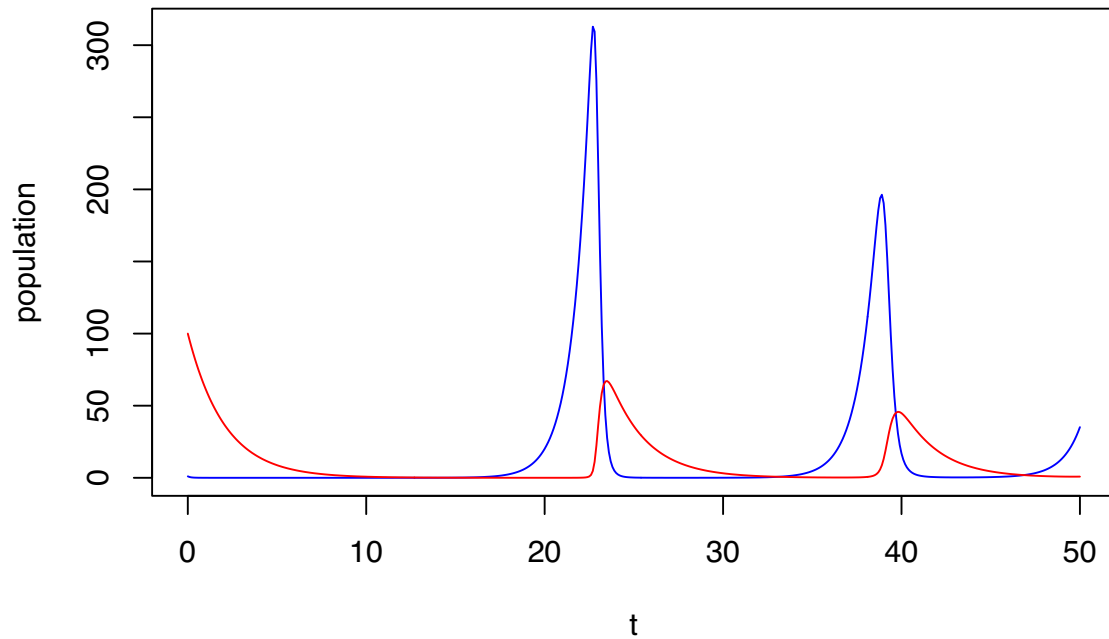
```
# [0, 25] with initial condition (0, 10)
df1 <- BE(25, c(0, 10))
tt <- seq(0, 25, by=h)
plot(tt, df1[1,], type='l', xlab="t", ylab="population", col='blue', main="Predator (red) v. Prey (blue)
lines(tt, df1[2,], col='red')
```

Predator (red) v. Prey (blue) Population over Time



```
# [0, 25] with initial condition (1, 100)
df1 <- BE(50, c(1, 100))
tt <- seq(0, 50, by=h)
plot(tt, df1[1,], type='l', xlab="t", ylab="population", col='blue', main="Predator (red) v. Prey (blue)")
lines(tt, df1[2,], col='red')
```

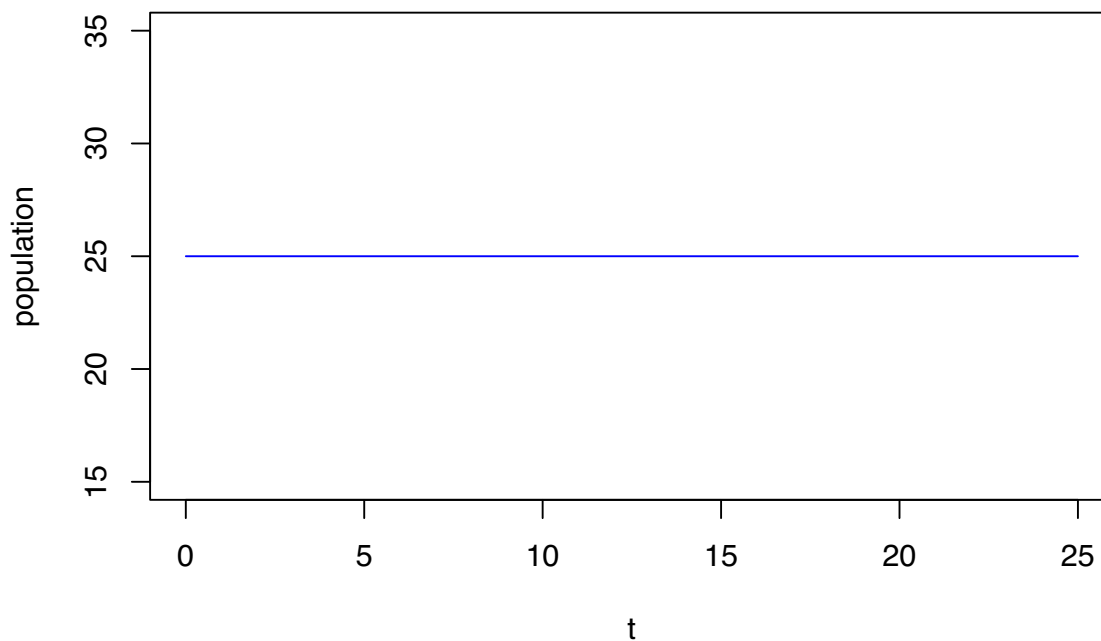
Predator (red) v. Prey (blue) Population over Time



Finally, observe the following steady state solution below:

```
# [0, 25] with initial condition (25, 10)
df1 <- BE(25, c(25, 10))
tt <- seq(0, 25, by=h)
plot(tt, df1[1,], type='l', xlab="t", ylab="population", col='blue', main="Predator (red) v. Prey (blue)
lines(tt, df1[2,], col='red')
```

Predator (red) v. Prey (blue) Population over Time



Why does this occur? Well, choosing $\vec{y}_0 = [25, 10]^\top$, observe that

$$J_f(0) = \begin{pmatrix} 1 - 0.1 \cdot 10 & -0.1 \cdot 25 \\ 10 \cdot 0.02 & -0.5 + 0.02 \cdot 25 \end{pmatrix} = \begin{pmatrix} 0 & -2.5 \\ 0.2 & 0 \end{pmatrix}$$

The eigenvalues of $J_f(0)$ are of the form $\lambda = 0 \pm bi$. Hence, $\vec{y}_0 = [25, 10]^\top$ is a center, or a fixed point of this dynamical system that cannot be achieved unless it is in fact the initial condition.